socode.

Programmation avec Python

Par: Nour Maafi

TABLE DES MATIÉRES

01 **Les Conditions**

Aperçu, if, else, elif

02 **Les Boucles**

Aperçu , While, For

03 **Structures de données**

Pourquoi les structures de données ?

04 **Les Listes**

La première de plusieurs…

01

Les Conditions

if, else, elif

Les Conditions

En Python, les **conditions** sont utilisées pour exécuter du code en fonction de certaines **expressions conditionnelles ou logiques**.

Les Expressions

En Python, les expressions logiques s’évaluent soit à **Vrai** (True), soit à **Faux** (False).

Les Expressions

En Python, les expressions logiques s’évaluent soit à **Vrai** (True), soit à **Faux** (False).

AND
« et »

OR
« Ou »

NOT
« Non »

Les Expressions

En Python, les expressions logiques s'évaluent soit à **Vrai** (True), soit à **Faux** (False).

AND « et »	OR « Ou »	NOT « Non »
X<3 and y>5	X<3 or y>5	not (y>5)

<<<<<

Les Expressions

En Python, les expressions mathématiques impliquent l’utilisation d’opérateurs arithmétiques pour effectuer des calculs.

Les Expressions

+: Addition

Exemple : 3+5 est 8

-: Soustraction

Exemple : 5-3 est 2

*: Multiplication

Exemple : 3*5 est 15

/: Division

Exemple : 9/2 est 4.5

//: Division entière

Exemple : 9//2 est 4

%: Modulo

Exemple : 9%2 est 1

**: Exponentiation

Exemple : 2**3 est 8

Logical expressions <<<<<

Types d’instructions conditionnelles

IF

if (condition) :

Teste une condition et exécute le bloc de code si la condition est Vraie

<<<<<

Types d’instructions conditionnelles

IF

if (condition) :

Teste une condition et exécute le bloc de code si la condition est Vraie

ELIF

elif (condition) :

Fournit des conditions supplémentaires à tester si les conditions if ou elif précédentes étaient Fausses.

<<<<<

Types d’instructions conditionnelles

IF

if (condition) :

Teste une condition et exécute le bloc de code si la condition est Vraie

ELIF

elif (condition) :

Fournit des conditions supplémentaires à tester si les conditions if ou elif précédentes étaient Fausses.

ELSE

else :

Exécute un bloc de code si aucune des conditions précédentes n’était Vraie.

<<<<<

Exercice 01

Écrivez un programme Python qui :

- *teste la saisie de l’utilisateur (âge) et lui indique s’il est mineur ou majeur.*

Exercice 02

Écrivez un programme Python qui :

- *teste la saisie (input) de l’utilisateur (nom d’utilisateur, mot de passe) avec des données prédéfinies et affiche le résultat.*

02

Les Boucles

While , For

Les Boucles

Les boucles sont utilisées pour exécuter un bloc de code de manière répétée tant qu'une certaine condition est remplie ou pour un nombre spécifique de fois.
Python possède deux principaux types de boucles : les boucles for et les boucles while .

La Boucle FOR

Une boucle for est utilisée pour itérer sur une séquence.

La Boucle FOR

Une boucle for est utilisée pour itérer sur une séquence.

Syntaxe :

```
for i in range(x):
    #do somthing
```

La Boucle FOR

Une boucle for est utilisée pour itérer sur une séquence.

Syntax :

```
for i in range(x):
    #do somthing
```

Exemple :

```
for i in range(5):
    print(i)
```

La Boucle FOR

Une boucle for est utilisée pour itérer sur une séquence.

Syntax :

```
for i in range(x):
    #do somthing
```

Example :

```
for i in range(5):
    print(i)
```

Range : la fonction de comptage permet de spécifier des valeurs de début, de fin et de pas.

range(start, stop, step)

Si **start** et **step** ne sont pas spécifiés, **start** sera 0 et **step** sera 1 par défaut.

<<<<<

La Boucle WHILE

Une boucle while continue de s’exécuter tant que sa condition est Vraie.

La Boucle WHILE

Une boucle while continue de s’exécuter tant que sa condition est Vraie.

Syntaxe :

```
while condition:
    #do somthing
```

La Boucle WHILE

Une boucle while continue de s’exécuter tant que sa condition est Vraie.

Syntaxe :

```
while condition:
    #do somthing
```

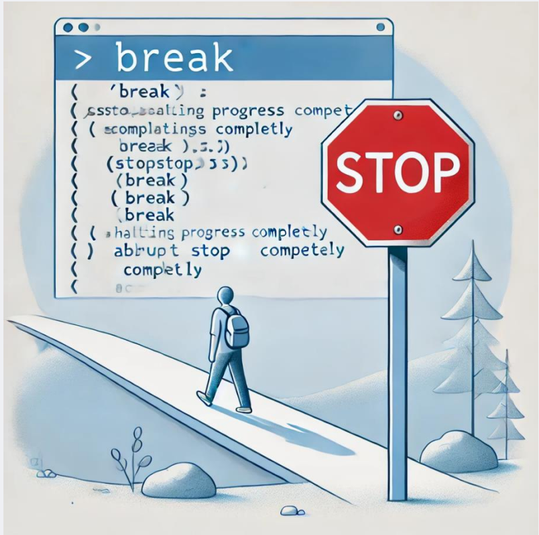
Exemple :

```
x = 0
while x < 5:
    print(x)
    x += 1
```

INSTRUCTIONS DE CONTRÔLE DE BOUCLE

Nous en avons trois

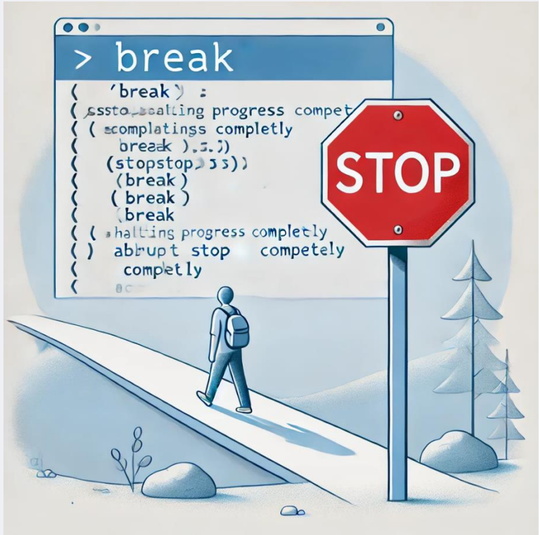
break :
Arrête la boucle



INSTRUCTIONS DE CONTRÔLE DE BOUCLE

Nous en avons trois

break :
Arrête la boucle



continue :
Ignore certaines étapes / passe à l'itération suivante

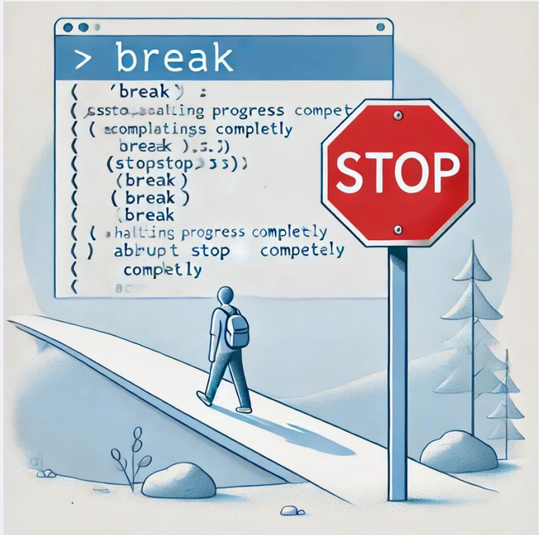
INSTRUCTIONS DE CONTRÔLE DE BOUCLE

Nous en avons trois

break :
Arrête la boucle

continue :
Ignore certaines étapes / passe à l'itération suivante

pass :
Ne fait rien, sert de **place-holder**



Exercice 01

Écrivez un programme Python qui :

- *Affiche les nombres de 0 à 10..*

Exercice 02

Écrivez un programme Python qui :

- *Lit la saisie (input) de l’utilisateur et l’affiche en majuscules, avec une lettre par ligne.*

03

Structures de données

Pourquoi ?

Structures de données

Les structures de données sont des moyens d'organiser, stocker et gérer les données dans un ordinateur afin qu'elles puissent être utilisées efficacement. Elles définissent la relation entre les données et fournissent les opérations nécessaires pour les manipuler.

Structures de données

Listes

Les listes sont des collections de données ordonnées et modifiables. Elles peuvent être utilisées pour stocker plusieurs éléments dans une seule variable.

Tuples

Les tuples sont des collections de données ordonnées mais immuables. Ils peuvent être utilisés pour stocker plusieurs éléments dans une seule variable, comme les listes, mais ne peuvent pas être modifiés une fois créés.

Ensembles

Les ensembles (sets) sont des collections non ordonnées de données uniques. Ils permettent de stocker des éléments sans duplication.

Dictionnaires

Les dictionnaires sont des collections non ordonnées de données sous forme de paires clé-valeur. Ils sont utilisés pour stocker des données qui peuvent être rapidement récupérées grâce à leur clé.

04

Les Listes

Les Listes

Les listes sont des collections de données ordonnées et modifiables.
Elles peuvent être utilisées pour stocker plusieurs éléments dans une seule variable.

Les Listes

Les listes sont des collections de données ordonnées et modifiables.
Elles peuvent être utilisées pour stocker plusieurs éléments dans une seule variable.

Créer

Les Listes

Les listes sont des collections de données ordonnées et modifiables.
Elles peuvent être utilisées pour stocker plusieurs éléments dans une seule variable.

Créer

Accéder

Les Listes

Les listes sont des collections de données ordonnées et modifiables.
Elles peuvent être utilisées pour stocker plusieurs éléments dans une seule variable.

Créer

Accéder

Modifier

Créer

```
# liste vide ( empty list )
empty_list = []
```

```
# Listes des entiers (list of integers )
numbers = [0, 1, 2, 3, 4, 5]
```

```
# Liste de types de données mixtes
mixed = [1, "Hello", 3.5, True]
```

```
# Liste imbriquée (une liste à
l’intérieur d’une liste)
nested = [[1, 2], [3, 4], [5, 6]]
```

Accéder

```
# Accéder à un élément unique par son index
my_list[0]

# Accéder à une plage d'éléments (découpage / slicing)
my_list[1:4]
my_list[:3]
my_list[2:]

# Accéder en utilisant une boucle
for item in my_list:
    print(item)
```

```
# Accéder aux éléments depuis la fin
my_list[-1]

# Accéder à chaque nième élément (slicing étendu)
my_list[::2]
my_list[::-1]

# Accéder en utilisant une compréhension de liste
squares = [x**2 for x in my_list]
print(squares)
```

Modifier

1. Mettre à jour un élément

```
my_list[1] = 25
```

Modifier

1. Mettre à jour un élément

```
my_list[1] = 25
```

2. Ajouter un élément

```
# Append (ajouter à la fin)
my_list.append(0)
```

```
# Insert (insérer à une position spécifique)
my_list.insert(2, 15)
```

```
# Extend (étendre avec une autre liste)
my_list.extend([60, 70])
```

<<<<<

Modifier

1. Mettre à jour un élément

```
my_list[1] = 25
```

2. Ajouter un élément

```
# Append (ajouter à la fin)
my_list.append(0)
```

```
# Insert (insérer à une position spécifique)
my_list.insert(2, 15)
```

```
# Extend (étendre avec une autre liste)
my_list.extend([60, 70])
```

3. Supprimer un élément

```
#Remove(supprimer par valeur)
my_list.remove(25)
```

```
# Supprimer par index
del my_list[2]
my_list.pop(1)
```

```
# Clear (vider la liste)
my_list.clear()
```

<<<<<

Modifier

4. Trier la liste

```
# Ordre croissant
my_list.sort()
```

```
# Ordre décroissant
my_list.sort(reverse=True)
```

Modifier

4. Trier la liste

```
# Ordre croissant
my_list.sort()
```

```
# Ordre décroissant
my_list.sort(reverse=True)
```

5. Inverser la liste

```
# Inverser l’ordre
my_list.reverse()
```

<<<<<

Modifier

4. Trier la liste

```
# Ordre croissant
my_list.sort()
```

```
# Ordre décroissant
my_list.sort(reverse=True)
```

5. Inverser la liste

```
# Inverser l’ordre
my_list.reverse()
```

6. Remplacer une portion

```
# Remplacer une portion
my_list[1:3] = [10, 20]
```

<<<<<

Exercice 01

Écrivez un programme Python qui :

- *Créez une liste de vos trois films préférés et affichez chaque film en utilisant une boucle for.*

Pause

Module Turtle

Le module turtle en Python est une bibliothèque intégrée qui offre un moyen simple de créer des graphiques, dessiner des formes et concevoir des motifs. Il est basé sur une "tortue" virtuelle qui se déplace sur l’écran en laissant une trace derrière elle.

Module Turtle

Le module turtle en Python est une bibliothèque intégrée qui offre un moyen simple de créer des graphiques, dessiner des formes et concevoir des motifs. Il est basé sur une "tortue" virtuelle qui se déplace sur l’écran en laissant une trace derrière elle.

Comment ça fonctionne ?

- Vous contrôlez un objet tortue en utilisant des commandes pour le déplacer vers l’avant, l’arrière, la gauche ou la droite.
- Lorsque la tortue se déplace, elle trace des lignes ou des formes à l’écran, créant ainsi des graphiques.

Comment ça fonctionne ?

- 1. Importer le module turtle

```
import turtle
```

Comment ça fonctionne ?

- 1. Importer le module turtle

```
import turtle
```

- 2. Créer un objet tortue

```
t = turtle.Turtle()
```

Comment ça fonctionne ?

- 1. Importer le module turtle

```
import turtle
```

- 2. Créer un objet tortue

```
t = turtle.Turtle()
```

- 3. Exécuter des commandes pour déplacer la
- tortue

```
t.forward(100)  # Avancer de 100 unites
t.right(90)     # Tourner à droite de 90 degrés
t.forward(50)   # Avancer de 50 unités
```

Comment ça fonctionne ?

- 1. Importer le module turtle

```
import turtle
```

- 2. Créer un objet tortue

```
t = turtle.Turtle()
```

- 3. Exécuter des commandes pour déplacer la
- tortue

```
t.forward(100)  # Move forward 100 units
t.right(90)     # Turn right 90 degrees
t.forward(50)   # Move forward 50 units
```

- 4. Afficher la fenêtre

```
turtle.done()
```

Commandes de base de Turtle

- turtle.forward(x) Déplacer la tortue vers l’avant de x unités.
- turtle.backward(x) Déplacer la tortue vers l’avant de x unités.
- turtle.right(angle) Tourner la tortue vers la droite de angle degrés.
- turtle.left(angle) Tourner la tortue vers la gauche de angle degrés.
- turtle.penup() Lever le crayon pour arrêter de dessiner pendant le déplacement.
- turtle.pendown() Abaisser le crayon pour recommencer à dessiner.
- turtle.goto(x, y) Déplacer la tortue à une position spécifique (x, y).
- turtle.circle(r) Dessiner un cercle de rayon r.
- turtle.color(c) Définir la couleur du crayon (c peut être un nom de couleur ou une valeur hex).
- turtle.speed(s) Définir la vitesse de la tortue (s varie de 0 à 10, ou "fast").
- turtle.showturtle() Afficher le curseur de la tortue.
- turtle.hideturtle() Masquer le curseur de la tortue.

Exercice 01

- **Utiliser turtle pour dessiner une étoile .**

Les Tuples

Les Tuples

Un tuple est une collection ordonnée et immuable d'éléments en Python. Il est similaire à une liste, mais ne peut pas être modifié après sa création. Les tuples sont couramment utilisés pour stocker des données qui ne doivent pas être modifiées, comme des coordonnées, des paramètres de configuration ou plusieurs valeurs liées.

Les Tuples

Un tuple est une collection ordonnée et immuable d'éléments en Python. Il est similaire à une liste, mais ne peut pas être modifié après sa création. Les tuples sont couramment utilisés pour stocker des données qui ne doivent pas être modifiées, comme des coordonnées, des paramètres de configuration ou plusieurs valeurs liées.

Créer

Les Tuples

Un tuple est une collection ordonnée et immuable d'éléments en Python. Il est similaire à une liste, mais ne peut pas être modifié après sa création. Les tuples sont couramment utilisés pour stocker des données qui ne doivent pas être modifiées, comme des coordonnées, des paramètres de configuration ou plusieurs valeurs liées.

Create

Accéder

Les Tuples

Un tuple est une collection ordonnée et immuable d'éléments en Python. Il est similaire à une liste, mais ne peut pas être modifié après sa création. Les tuples sont couramment utilisés pour stocker des données qui ne doivent pas être modifiées, comme des coordonnées, des paramètres de configuration ou plusieurs valeurs liées.

Créer

Accéder

Modifier

Les Tuples

Un tuple est une collection ordonnée et immuable d'éléments en Python. Il est similaire à une liste, mais ne peut pas être modifié après sa création. Les tuples sont couramment utilisés pour stocker des données qui ne doivent pas être modifiées, comme des coordonnées, des paramètres de configuration ou plusieurs valeurs liées.

Créer

Accéder

Opérer

Créer

```
# Tuple vide
empty_tuple = ()
```

```
# Tuple des entiers
numbers = (1, 2, 3, 4, 5)
Simpler_tuple = 1,2,3,4,5
```

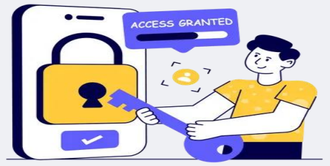
```
# Des valeurs mixes
mixed= (1,True,”SoCode”,4.5)
```

```
# Attention !
Tuple = (5,)
Variable = (5)
```

Accéder

```
# Accéder à un élément unique par son index
my_tuple[0]

# Accéder à une plage d'éléments (découpage / slicing)
my_ tuple[1:4]
my_ tuple[:3]
my_ tuple[2:]

# Accéder en utilisant une boucle
for item in my_ tuple :
    print(item)
```

```
# Accéder aux éléments depuis la fin
my_ tuple[-1]

# Accéder à chaque nième élément (slicing étendu)
my_ tuple[::2]
my_ tuple[::-1]

# Accéder en utilisant une compréhension de liste
squares = [x**2 for x in my_ tuple]
print(squares)
```



Opérer

1. Operations

```
# fusion
 result = tuple1 + tuple2
```

```
# repetition
my_tuple * 3
```

2. Methodes

```
# retourne le nombres des
elemenets du tuple = 20
my_tuple.count(20)
```

```
# retourner le premier index de
la valeur  20
my_tuple.index(20)
```

<<<<<

Exercice 01

- *Créez un tuple avec les noms des jours de la semaine et demandez à l’utilisateur de saisir une tâche pour chaque jour. Affichez la tâche pour chaque jour avec le jour correspondant de la semaine.*

Les Ensembles

Les Ensembles

Les ensembles sont des collections non ordonnées d'éléments uniques. Cela signifie qu'il n'y a aucun doublon dans un ensemble et que les éléments n'ont pas d'ordre particulier.

Les Ensembles

Les ensembles sont des collections non ordonnées d'éléments uniques. Cela signifie qu'il n'y a aucun doublon dans un ensemble et que les éléments n'ont pas d'ordre particulier.

Créer

Les Ensembles

Les ensembles sont des collections non ordonnées d'éléments uniques. Cela signifie qu'il n'y a aucun doublon dans un ensemble et que les éléments n'ont pas d'ordre particulier.

Créer

Accéder

Les Ensembles

Les ensembles sont des collections non ordonnées d'éléments uniques. Cela signifie qu'il n'y a aucun doublon dans un ensemble et que les éléments n'ont pas d'ordre particulier.

Créer

Accéder

Modifier/Opérer

Créer

```
# Ensemble Vide
empty_set = {}
```

```
# Ensemble des entiers
numbers = {1, 2, 3, 4, 5}
Constructor_set =
set([1,2,3,4,5])
```

```
# Des valeurs mixes
mixed= {1,True,”SoCode”,4.5}
```

+
+
+
+
+

<<<<<

Accéder

Supprime et renvoie un élément arbitraire de l’ensemble. Comme les ensembles ne sont pas ordonnés, on ne peut pas prédire quel élément sera supprimé.

```
random_element = fruits.pop()
```

Opérer

1. Méthodes

```
# Ajouter un élement
fruits.add("orange")
```

```
# remove (provoque une erreur)
fruits.remove("banana")
```

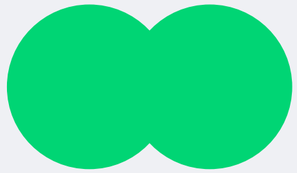
```
# discard un élément
fruits.discard("grape")
```

```
# supprimer tous les éléments
fruits.clear()
```

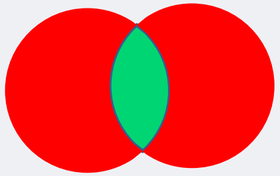
Opérer

2. Opérations

```
# union de deux ensembles
union_set = set1 | set2
```

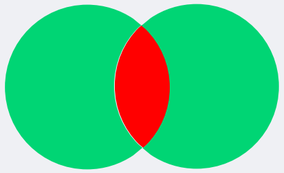


```
# intersection de deux ensembles
intersection_set = set1 & set2
```



```
+
+
+ # difference de deux ensembles
+ difference_set = set1 - set2
+
```

```
Symmetric difference of two sets
symmetric_difference_set = set1 ^ set2
```



<<<<<

Exercice 01

•Vous gérez une petite base de données d’étudiants pour une école. L’école propose plusieurs cours, et chaque cours a un ensemble d’étudiants inscrits. Vous allez effectuer une série d’opérations en utilisant des ensembles pour gérer les étudiants de ces cours.

Étapes:

•Créer des ensembles pour les cours :

Créez un ensemble d’étudiants inscrits dans deux cours : Math et Physique.

Les étudiants sont représentés par leurs noms sous forme de chaînes de caractères.

Ajouter des étudiants :

Ajoutez un nouvel étudiant à chaque cours. Ajoutez un nouvel étudiant à Math et un autre à Physique.

•Supprimer des étudiants :

Un étudiant a décidé d’abandonner Physique. Supprimez cet étudiant de l’ensemble Physique.

•Trouver les étudiants inscrits dans les deux cours (Intersection) :

Trouvez l’ensemble des étudiants qui sont inscrits à la fois en Math et en Physique (intersection des deux ensembles).

•Trouver les étudiants inscrits dans l’un ou l’autre des cours (Union) :

Trouvez l’ensemble des étudiants qui sont inscrits soit en Math, soit en Physique (union des deux ensembles).

•Trouver les étudiants inscrits en Math mais pas en Physique (Différence) :

Trouvez l’ensemble des étudiants inscrits en Math mais pas en Physique (différence des deux ensembles)..

•Trouver les étudiants inscrits dans un seul cours mais pas les deux (Différence symétrique) :

Trouvez les étudiants inscrits en Math ou en Physique, mais pas dans les deux cours.

•Vérifier si un étudiant est dans un cours :

Vérifiez si un étudiant spécifique (par exemple, "Alice") est inscrit en Math..

•Exemple de données :

•Math Students: "John", "Alice", "Bob", "Charlie"

•Physics Students: "Alice", "David", "Charlie", "Emma"

Les Dictionnaires

Les Dictionnaires

En Python, un dictionnaire est une collection non ordonnée de paires clé-valeur. C’est une structure de données polyvalente qui permet de stocker des données où chaque valeur est associée à une clé unique. Les dictionnaires sont également appelés **maps** ou **hash maps** dans d’autres langages de programmation.

Les Dictionnaires

En Python, un dictionnaire est une collection non ordonnée de paires clé-valeur. C’est une structure de données polyvalente qui permet de stocker des données où chaque valeur est associée à une clé unique. Les dictionnaires sont également appelés **maps** ou **hash maps** dans d’autres langages de programmation.

Créer

Les Dictionnaires

En Python, un dictionnaire est une collection non ordonnée de paires clé-valeur. C’est une structure de données polyvalente qui permet de stocker des données où chaque valeur est associée à une clé unique. Les dictionnaires sont également appelés **maps** ou **hash maps** dans d’autres langages de programmation.

Créer

Accéder

Les Dictionnaires

En Python, un dictionnaire est une collection non ordonnée de paires clé-valeur. C’est une structure de données polyvalente qui permet de stocker des données où chaque valeur est associée à une clé unique. Les dictionnaires sont également appelés **maps** ou **hash maps** dans d’autres langages de programmation.

Créer

Accéder

Modifier/Opérer

Créer

```
student = {
"name": "John",
"age": 21,
"course": "Computer Science" }
```

<<<<<

Accéder

Pour accéder à la valeur associée à une clé spécifique, utilisez la notation entre crochets [].

```
student["name"]
```

```
# Si n’existe pas, une nouvelle paire clé-valeur sera créée.
student["name"] = value
```

Opérer

1. Méthodes

```
# Supprimer la clé email
et sa valeur

del student["email"]
```

```
# Supprimer la clé age et
retourner sa valeur

student.pop("age")
```

+
+
+
+
+

```
# Parcourir les clés et les valeurs
for key in student:
print(key)
```

```
# Parcourir les paires clé-valeur
for key, value in student.items():
print(key, ":", value)
```

<<<<<

Opérer

2. Opérations

# Retourner toutes les clés
student.keys()

Key : value

# Retourner toutes les valeurs
student.values()

Key : value

# Retourner les clés et les valeurs
student.items()

Key : value

# Retourner la valeur de email si elle existe, sinon retourner « Non disponible »
student.get("email", "Not Available")

Key : value (if key exists)

<<<<<

Bonus

Retourner la longueur d’une liste, d’un ensemble (set), d’un tuple ou d’un dictionnaire

len(fruits)

Retourner si l’élément « apple » existe dans une liste, un ensemble (set), un tuple ou un dictionnaire

"apple" in fruits

Exercice 01

•Vous gérez une liste d’étudiants et leurs notes dans un dictionnaire. Votre tâche est de :

1.Créer un dictionnaire avec les noms des étudiants comme clés et leurs notes comme valeurs.

2.Ajouter un nouvel étudiant et sa note.

3.Mettre à jour la note d’un étudiant existant.

4.Supprimer un étudiant du dictionnaire.

5.Vérifier si un étudiant existe dans le dictionnaire.

6.Afficher tous les étudiants et leurs notes.

7.Data:

•"John" a une note de 85.

•"Alice" a une note de 90.

•"Bob" a une note de 78.